# RAPPORT D'ANALYSE FORENSIC

*Acquisition Mémoire & Disque*

TP Forensic – Partie I.I

| | |
|---|---|
| **Analyste** | Nowak Patrick |
| **Groupe** | B3 Cyber |
| **Date de l'analyse** | 24/06/2025 |
| **Machine d'analyse** | Kali Linux – QEMU / Architecture arm64 |
| **Machine cible** | Kali Linux adaptée pour le TP |
| **Hyperviseur** | QEMU Virtual Machine |
| **Dossier de travail** | ~/Desktop/EVIDENCES_PART_I.I |
| **Kernel** | Linux 6.16.8+kali-arm64 |
| **Version du rapport** | 1.0 – Rapport initial |
| **Statut** | Finalisé |

*Document pédagogique – Usage interne uniquement. Ne pas diffuser.*

# Table des matières

1. Préambule et contexte de l'investigation ..... 3
2. Environnement d'analyse et configuration du laboratoire ..... 4
3. Fondements théoriques — Images mémoire ..... 5
4. Fondements théoriques — Images disque ..... 6
5. Inventaire des outils forensics ..... 7
6. Acquisition RAM depuis une VM VMware (méthode VMEM) ..... 8
7. Snapshot VMware comme alternative d'acquisition ..... 9
8. Critères de fiabilité d'une acquisition mémoire ..... 10
9. Adaptation Linux — Observation de la mémoire sur Kali ..... 10
10. Acquisition RAM avec DumpIT (méthode Windows) ..... 12
11. Acquisition RAM avec FTK Imager (méthode Windows) ..... 13
12. Acquisition disque — Équivalent dd sur Kali ..... 14
13. Vérification d'intégrité des images forensics ..... 16
14. Parties optionnelles ..... 17
15. Synthèse générale et chaîne de conservation ..... 18
16. Conclusion et enseignements ..... 19
Annexe A Lexique forensic ..... 20
Annexe B Commandes de référence ..... 21

# 1. Préambule et contexte de l'investigation

## 1.1 Objectif du travail pratique

Ce rapport documente l'ensemble des travaux réalisés dans le cadre du TP Forensic — Partie I.I, dont l'objectif est de comprendre et de pratiquer les principales méthodes d'acquisition forensic de mémoire vive (RAM) et de mémoire de masse (disque). L'acquisition forensic constitue le premier maillon de toute investigation numérique : sans une copie fidèle, intègre et documentée du support à analyser, aucune conclusion ne peut être considérée comme fiable ou admissible.

Le TP original est conçu pour un environnement Windows avec les outils DumpIT, FTK Imager et RedLine. Pour des raisons techniques liées à l'environnement disponible (machine Kali Linux sous QEMU, architecture arm64), une adaptation a été réalisée : les étapes Windows sont documentées de façon théorique et méthodologique, tandis que les acquisitions pratiques sont reproduites avec les équivalents Linux (dd, /proc/kcore, hostnamectl, etc.).

> **ADAPTATION LINUX / KALI**
> Cette adaptation est documentée et justifiée tout au long du rapport. Chaque section indique clairement si la procédure a été réalisée directement ou documentée en équivalence.

## 1.2 Importance de la prise de notes en forensic

La digital forensic est une discipline qui exige une rigueur documentaire absolue. Contrairement à d'autres domaines de l'informatique où les actions peuvent être refaites, une investigation forensic porte sur un état du système à un instant donné. Toute action non documentée est une action qui ne pourra pas être justifiée ultérieurement, que ce soit devant un tribunal, face à un client ou lors d'une revue entre pairs.

Les cinq impératifs de documentation en forensic numérique sont :

- **Chronologie :** chaque action est horodatée et consignée dans l'ordre d'exécution.
- **Reproductibilité :** chaque commande est notée avec ses paramètres exacts, permettant à un tiers de reproduire l'analyse.
- **Traçabilité :** la chaîne de conservation (chain of custody) documente le parcours de chaque preuve de sa collecte à sa présentation.
- **Intégrité :** les hashes cryptographiques (MD5, SHA1, SHA256) sont calculés à chaque étape critique pour démontrer l'absence de modification.
- **Exhaustivité :** les observations, erreurs, déviations de procédure et décisions de l'analyste sont toutes consignées, y compris les éléments négatifs.

Les outils forensics professionnels référencés dans le sujet qui produisent des rapports structurés intégrant ces éléments incluent notamment : Autopsy (open source), X-Ways Forensics, EnCase et Maltego.

# 2. Environnement d'analyse et configuration du laboratoire

## 2.1 Architecture du laboratoire

Le laboratoire forensic décrit dans le sujet de TP comprend plusieurs machines virtuelles aux rôles distincts. Cette architecture multi-machines reflète un environnement d'investigation réaliste, où la machine d'analyse doit être clairement séparée des machines cibles afin d'éviter toute contamination croisée.

| Machine | Rôle | Description |
|---|---|---|
| `CAINE` | Machine d'analyse forensic | Poste de travail de l'analyste — outils forensics installés |
| `LINUX-FORENSICS` | Première machine cible | Système Linux à analyser |
| `TARGET-FORENSICS` | Cible avec fichiers mémoire | Contient des images mémoire à analyser |
| `MALWARE-FORENSICS` | Machine Windows compromise | Simulation d'incident de sécurité |
| `Kali Linux (TP)` | Machine utilisée en remplacement | Adaptation technique de l'environnement |

## 2.2 Configuration de la machine utilisée

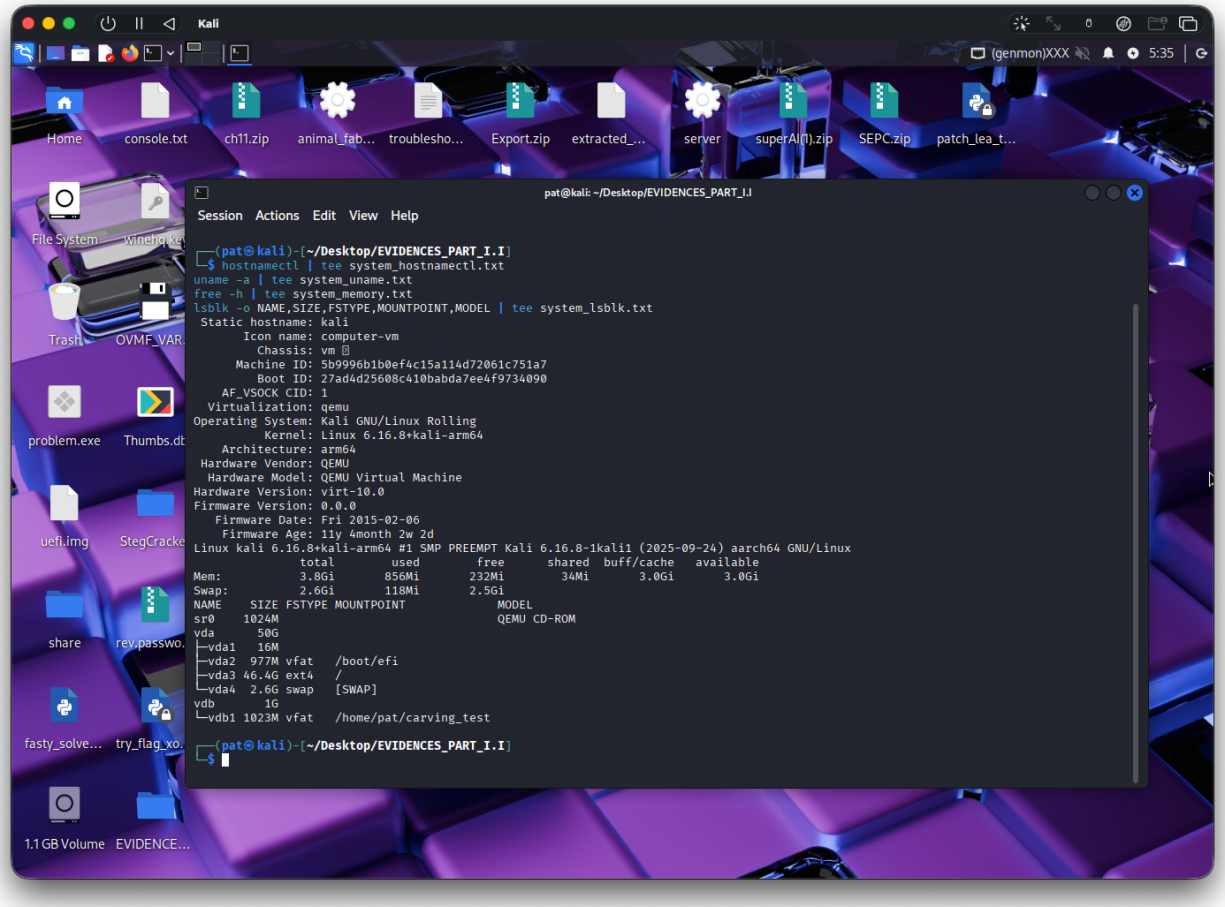
Le TP est réalisé sur une machine Kali Linux virtualisée sous QEMU, architecture ARM 64 bits. Les informations système détaillées, obtenues lors de la phase d'observation (section 9), sont les suivantes :

| | |
|---|---|
| **Hostname** | `kali` |
| **Système** | Kali GNU/Linux Rolling |
| **Kernel** | `Linux 6.16.8+kali-arm64` |
| **Architecture** | arm64 / aarch64 |
| **Virtualisation** | QEMU Virtual Machine |
| **RAM totale** | 3,8 GiB (4 004 288 kB selon /proc/meminfo) |
| **RAM utilisée** | 825 MiB |
| **Swap total** | 2,6 GiB (2 709 500 kB) |
| **Dossier de travail** | `~/Desktop/EVIDENCES_PART_I.I` |

## 2.3 Recommandations préalables à toute investigation

Avant de démarrer une investigation sur une machine virtuelle, les bonnes pratiques suivantes doivent être appliquées :

- Augmenter les ressources CPU et RAM de la VM si les outils d'analyse sont gourmands en ressources.
- Vérifier l'espace disque disponible sur la machine d'analyse avant l'acquisition — une image disque de 1 Go nécessite au minimum 1 Go libre, plus l'espace pour les outils et les fichiers temporaires.
- Réaliser un snapshot de la machine d'analyse avant toute installation de nouvel outil, afin de pouvoir revenir à un état propre.
- Ne jamais démarrer l'analyse sur le support original — toujours travailler sur une copie forensic vérifiée.




# 3. Fondements théoriques — Images mémoire

## 3.1 Intérêt forensic de l'acquisition mémoire

La mémoire vive (RAM) est le composant le plus volatil d'un système informatique. Elle contient l'intégralité de l'état d'exécution du système à un instant donné : les processus en cours, les connexions réseau établies, les clés de chiffrement chargées, les mots de passe en clair dans certains cas, les commandes récentes et les artefacts d'exécution de malwares. Dès que l'alimentation est coupée ou que le système est redémarré, ces informations sont définitivement perdues.

En investigation forensic, l'acquisition mémoire permet de capturer cet état volatile pour analyse ultérieure. Elle est particulièrement précieuse dans les scénarios suivants :

- Investigation sur un incident de sécurité actif : un malware en mémoire uniquement (fileless malware) ne laisse aucune trace sur le disque.
- Analyse de ransomware : les clés de chiffrement peuvent être présentes en RAM pendant le processus de chiffrement.
- Investigation sur une session RDP ou VPN active : les credentials peuvent se trouver en mémoire.
- Analyse d'un processus injecté ou d'une technique de process hollowing.
- Recherche de connexions réseau établies, de sockets ouverts, de tables ARP et de routes en mémoire.

## 3.2 Formats d'images mémoire

Plusieurs formats d'images mémoire sont utilisés selon l'outil d'acquisition et le système d'exploitation cible. Chaque format présente des caractéristiques propres qui influencent sa compatibilité avec les outils d'analyse.

| Format | Extension | Type | Description forensic |
|---|---|---|---|
| RAW | .raw / .bin | Binaire brut | Copie bit-à-bit de la RAM. Simple et universel. Compatible avec la plupart des outils d'analyse dont Volatility. Pas de métadonnées intégrées. |
| DMP | .dmp | Windows Dump | Format dump mémoire natif Windows. Produit par DumpIT ou lors de crashes système (BSOD). Contient des en-têtes structurés. |
| VMEM | .vmem | VMware Memory | Fichier généré automatiquement par VMware lors de la mise en pause d'une VM. Représente l'état de la RAM de la machine virtuelle. |
| LiME | .lime | Linux Memory | Format spécifique au module noyau LiME (Linux Memory Extractor). Inclut des en-têtes par segments de mémoire physique. |
| Crash Dump | .dmp | Windows | Dump généré lors d'un arrêt inopiné Windows (BSOD). Peut être complet (full), kernel-only ou mini-dump selon la configuration. |
| Hibernation | hiberfil.sys | Windows | Fichier créé lors de l'hibernation Windows. Contient la RAM compressée. Analysable avec des outils spécialisés. |

**NOTE FORENSIC**

Pour l'analyse post-acquisition, Volatility Framework est l'outil de référence open source, compatible avec la majorité des formats ci-dessus. Il permet d'extraire processus, connexions réseau, ruches de registre, etc. depuis une image mémoire.

# 4. Fondements théoriques — Images disque

## 4.1 Intérêt forensic de l'image disque

L'acquisition d'une image disque (ou image de partition) produit une copie bit-à-bit du support de stockage, incluant les espaces libres, les secteurs supprimés, les zones de slack space et les structures de bas niveau du système de fichiers. Cette copie fidèle permet de travailler sur un artefact immuable et de répéter les analyses sans risque d'altérer les données originales.

Contrairement à l'acquisition mémoire, l'image disque capture des données persistantes : fichiers actifs, fichiers supprimés récupérables, journaux de système de fichiers, métadonnées (timestamps, permissions, attributs), et artefacts de systèmes de fichiers.

## 4.2 Formats d'images disque

| Format | Extension | Type | Description forensic |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **RAW** | .img / .dd / .raw | Binaire brut | Copie bit-à-bit universelle. Simple, large compatibilité. Pas de compression ni de métadonnées intégrées. Produit par dd. |
| **E01 / EWF** | .E01 / .Ex01 | Expert Witness | Format forensic de référence. Compression, métadonnées (analyste, cas, hash), découpage en segments. Produit par FTK Imager, EnCase. |
| **AFF** | .aff / .afd | Advanced Forensic | Format forensic open source avec métadonnées extensibles et compression. Successeur amélioré du RAW. Produit par afftools. |
| **FTK Smart** | .s01 | Propriétaire | Format propriétaire AccessData/Exterro. Compression et métadonnées. Utilisé dans l'écosystème FTK. |
| **VMDK** | .vmdk | VMware Virtual Disk | Disque virtuel VMware. Analysable directement comme preuve sans acquisition sur la machine invitée. |
| **AD1** | .ad1 | AccessData Custom | Conteneur FTK pour collections de fichiers et dossiers. Conserve les métadonnées et hashes. |

**Avantages du format E01/EWF sur RAW :** le format E01 intègre directement dans le fichier image les métadonnées de l'analyste, le hash de vérification, la date d'acquisition et d'éventuelles notes de cas. Ces informations sont indissociables de l'image, ce qui renforce la valeur probatoire de l'artefact. Le format RAW, plus simple et universel, est préféré dans les environnements Linux où dd est l'outil standard.

# 5. Inventaire des outils forensics

## 5.1 Tableau comparatif des outils

Le tableau ci-dessous recense les principaux outils d'acquisition et d'analyse forensic mentionnés dans le sujet, complétés par des informations sur leur licence, leur système cible et leur format de sortie. Une note importante sur la classification : le sujet mélange parfois les notions d'open source, de gratuit et de propriétaire. Ce tableau clarifie ces distinctions.

| Catégorie | Outil | Licence | OS cible | Format de sortie | Usage principal |
|---|---|---|---|---|---|
| RAM | LiME (Linux Memory Extractor) | Open source (GPL) | Linux / Android | RAW, Lime | Acquisition kernel module |
| RAM | DumpIT | Gratuit (MoonSols) | Windows x86/x64 | RAW / DMP | Acquisition one-click |
| RAM | MemProcFS | Open source | Windows / Linux | Fichiers virtuels | Navigation mémoire en FUSE |
| RAM | Volatility Framework | Open source | Analyse (multi-OS) | Analyse seule | Analyse post-acquisition |
| RAM | RAMMap | Gratuit (Sysinternals) | Windows | Lecture seule | Cartographie RAM Windows |
| RAM | RedLine | Gratuit (Mandiant) | Windows | Sessions / AD1 | Audit & triage malware |
| RAM | FTK Imager (RAM) | Gratuit (Exterro) | Windows | MEM + AD1 | Acquisition GUI Windows |
| Disque | dd (GNU) | Open source (GPL) | Linux / macOS | RAW | Copie bit-à-bit universelle |
| Disque | Autopsy / TSK | Open source | Multi-OS | Analyse seule | Analyse de systèmes de fichiers |
| Disque | FTK Imager (disque) | Gratuit (Exterro) | Windows | E01 / RAW / AD1 | Acquisition GUI Windows |
| Disque | AccessData FTK | Propriétaire (payant) | Windows | E01 / AFF | Suite forensic complète |
| Disque | EnCase | Propriétaire (payant) | Windows | E01 | Suite forensic judiciaire |
| Disque | X-Ways Forensics | Propriétaire (payant) | Windows | Multi-formats | Analyse avancée |

> ⚠ **POINT D'ATTENTION**
> Volatility Framework est un outil d'ANALYSE et non d'acquisition. Il ne produit pas de dump mémoire : il analyse des images déjà acquises par d'autres outils (LiME, DumpIT, FTK Imager, etc.). Cette distinction est fondamentale pour comprendre la chaîne d'acquisition forensic.

## 5.2 Choix des outils dans ce TP

Dans le cadre de ce TP réalisé sur Kali Linux / QEMU, les équivalents Linux suivants ont été utilisés en substitution aux outils Windows :

| | |
|---|---|
| **DumpIT (Windows)** | → /proc/kcore + LiME (observation Linux) |

| | |
|---|---|
| **FTK Imager RAM (Windows)** | → Observation /proc/meminfo + free -h |
| **FTK Imager Disque (Windows)** | → dd (acquisition bit-à-bit Linux) |
| **certutil (Windows)** | → sha256sum (Linux) |

# 6. Acquisition RAM depuis une VM VMware — Méthode VMEM

## 6.1 Principe du fichier VMEM

VMware Workstation / Fusion génère automatiquement un fichier .vmem lorsqu'une machine virtuelle est mise en pause (suspend) ou lorsqu'un snapshot incluant la mémoire est créé. Ce fichier représente l'état exact de la RAM de la machine virtuelle à l'instant de la mise en pause — il s'agit donc d'une image mémoire prête à l'emploi, sans aucun outil d'acquisition supplémentaire côté invité.

Cette technique présente un avantage forensic considérable : l'acquisition se fait côté hyperviseur, sans interaction avec le système d'exploitation invité. Elle ne laisse donc aucune trace dans la RAM ou les journaux de la machine analysée, contrairement à l'exécution d'un outil comme DumpIT qui modifie inévitablement l'état de la mémoire en s'exécutant.

## 6.2 Structure du dossier d'une VM VMware

Les fichiers constitutifs d'une machine virtuelle VMware et leur rôle forensic sont les suivants :

| | |
|---|---|
| **.vmx** | Fichier de configuration de la VM (texte). Contient les paramètres matériels, réseau, et les chemins des fichiers associés. |
| **.vmdk** | Disque virtuel de la VM. Peut être analysé directement comme image disque sans acquisition. |
| **.vmem** | Image mémoire de la VM lors d'une pause ou d'un snapshot avec mémoire. Analysable avec Volatility. |
| **.vmss** | État suspendu de la VM (suspend state). Complète le .vmem lors d'une suspension. |
| **.vmsn** | Snapshot de la VM. Contient les différences depuis le snapshot précédent. |
| **.nvram** | BIOS/NVRAM virtuel de la VM. |

## 6.3 Procédure d'acquisition du fichier VMEM

La procédure pour récupérer le fichier .vmem dans un contexte forensic sur VMware est la suivante. Elle ne nécessite aucun outil spécifique côté machine invitée.

1. Démarrer la VM cible et réaliser si nécessaire les actions simulant l'activité à investiguer.

2. Mettre la VM en pause (Suspend) depuis l'interface VMware. Cette action écrit immédiatement l'état de la RAM dans le fichier .vmem.
3. Ouvrir le dossier de la VM sur le système hôte.
4. Identifier le fichier .vmem (son nom reprend généralement le nom de la VM).
5. Copier le fichier .vmem dans le dossier d'évidences dédié au cas.
6. Calculer et archiver le hash SHA256 immédiatement après la copie.

**Calcul du hash SHA256 du fichier VMEM**

```
# Sur un hôte Linux/macOS : liste et hash du fichier .vmem
ls -lh /chemin/vers/VM/

# Calcul du hash SHA256 (Linux/macOS)
sha256sum NOM_VM.vmem | tee NOM_VM.vmem.sha256

# Sur un hôte Windows (PowerShell)
certutil -hashfile "NOM_VM.vmem" SHA256
Get-FileHash -Path "NOM_VM.vmem" -Algorithm SHA256
```

| | |
|---|---|
| **Nom du fichier VMEM** | À compléter par l'analyste |
| **Chemin complet** | À compléter par l'analyste |
| **Hash SHA256** | À compléter par l'analyste |
| **Taille** | À compléter par l'analyste |

📷 **CAPTURE 2 — Dossier de la VM montrant le fichier .vmem**

Commande(s) à capturer :

```
ls -lh /chemin/dossier/VM/
dir (Windows)
```

📷 **CAPTURE 3 — Hash SHA256 du fichier .vmem calculé et archivé**

Commande(s) à capturer :

```
sha256sum NOM_VM.vmem | tee NOM_VM.vmem.sha256
```

# 7. Snapshot VMware comme alternative d'acquisition

## 7.1 Principe et utilité forensic

Le snapshot VMware est une fonctionnalité qui capture l'état complet d'une machine virtuelle à un instant donné : l'état du disque, la configuration, et optionnellement l'état de la mémoire vive si l'option correspondante est cochée lors de la création du snapshot.

Dans un contexte forensic, le snapshot avec mémoire présente un avantage important par rapport à la suspension complète : il permet de capturer l'état de la RAM sans interrompre durablement le fonctionnement de la machine. Le système invité continue à s'exécuter après la création du snapshot, tandis que le fichier .vmem du snapshot est disponible pour analyse.

## 7.2 Avantages et limites

| ✓ Avantages | ✖ Limites |
|---|---|
| ▪ Ne suspend pas durablement la VM cible<br>▪ Accessible depuis l'hyperviseur sans intervention côté invité<br>▪ Capture disque + mémoire en une seule opération<br>▪ Peut être automatisé dans un script de réponse à incident<br>▪ Le .vmem généré est directement analysable avec Volatility | ▪ Nécessite un accès à l'hyperviseur (pas toujours disponible)<br>▪ Requiert de l'espace disque suffisant sur l'hôte<br>▪ La création du snapshot modifie légèrement l'état de la VM<br>▪ Non applicable sur un système physique (bare metal)<br>▪ Peut être détecté par certains malwares anti-VM |

**CAPTURE 4 — Création d'un snapshot VMware avec l'option mémoire cochée**

Commande(s) à capturer :

```
VM > Snapshot > Take Snapshot > cocher 'Capture the virtual machine's memory'
```

# 8. Critères de fiabilité d'une acquisition mémoire

Une acquisition mémoire est considérée comme fiable et exploitable d'un point de vue forensic si elle satisfait trois critères fondamentaux, définis dans la littérature forensic et les standards du domaine :

| | |
|---|---|
| **Correcte** | L'image mémoire contient des données exactes, fidèles à l'état réel de la RAM au moment de l'acquisition. Tout artefact introduit par l'outil d'acquisition doit être connu et documenté. |
| **Atomique** | L'image n'est pas affectée par des activités concurrentes pendant l'acquisition. Sur un système en cours d'exécution, la RAM change en permanence (TOCTOU — Time Of Check vs Time Of Use). L'idéal est une capture instantanée, obtenue par exemple via un hyperviseur. |
| **Intègre** | L'outil d'acquisition ne modifie pas ou modifie le moins possible l'état de la mémoire analysée. Tout outil s'exécutant en mémoire modifie inévitablement son propre espace d'adressage. Cette modification doit être minimisée et documentée. |

**NOTE FORENSIC**

L'acquisition via hyperviseur (fichier .vmem de VMware, snapshot avec mémoire) est la méthode qui satisfait le mieux le critère d'atomicité et d'intégrité : la capture est réalisée depuis l'extérieur du système sans qu'aucun code s'exécute dans la mémoire de la VM.

# 9. Adaptation Linux — Observation de la mémoire sur Kali

## 9.1 Contexte de l'adaptation

**ADAPTATION LINUX / KALI**

DumpIT et FTK Imager RAM sont des outils conçus exclusivement pour Windows. L'environnement disponible étant Kali Linux sous QEMU (architecture arm64), ces outils n'ont pas pu être exécutés. Cette section documente l'équivalent Linux réalisé : observation des informations mémoire système, du pseudo-fichier /proc/kcore, et documentation de l'état du système.

Cette adaptation est valide d'un point de vue pédagogique : elle démontre la compréhension des mécanismes d'accès à la mémoire sous Linux et illustre les différences fondamentales entre les approches Windows et Linux en termes d'acquisition mémoire.

## 9.2 Informations système collectées

Les commandes suivantes ont été exécutées pour documenter l'état du système et simuler la phase d'identification préalable à toute acquisition forensic. Les résultats ont été archivés dans le dossier d'évidences.

**Commandes d'observation mémoire — Kali Linux**

```
# Identification complète du système
hostnamectl | tee system_hostnamectl.txt

# Informations noyau et architecture
uname -a | tee system_uname.txt

# État de la mémoire vive et du swap
free -h | tee system_memory.txt

# Détail complet de /proc/meminfo (30 premières lignes)
cat /proc/meminfo | head -30 | tee memory_proc_meminfo.txt

# Taille et type de /proc/kcore
ls -lh /proc/kcore | tee memory_kcore_info.txt
sudo file /proc/kcore | tee memory_kcore_file.txt
```

```
── hostnamectl ──────────────────────────────────────
 Static hostname: kali
 Virtualization: qemu
 Operating System: Kali GNU/Linux Rolling
 Kernel: Linux 6.16.8+kali-arm64
 Architecture: arm64
 Hardware Model: QEMU Virtual Machine

── free -h ──────────────────────────────────────────
               total    used    free   shared  buff/cache  available
 Mem:           3.8Gi  825Mi   477Mi   16Mi    2.7Gi       3.0Gi
 Swap:          2.6Gi  118Mi   2.5Gi

── /proc/kcore ──────────────────────────────────────
-r-------- 1 root root 4.0P ... /proc/kcore
/proc/kcore: ELF 64-bit LSB core file, ARM aarch64
```

## 9.3 Tableau récapitulatif de l'état mémoire

| | |
|---|---|
| **RAM totale** | 3,8 GiB (4 004 288 kB — /proc/meminfo MemTotal) |
| **RAM libre** | 370 472 kB (MemFree) |
| **RAM disponible** | 3 159 356 kB (~3,0 GiB — MemAvailable) |
| **RAM utilisée** | 825 MiB |
| **Swap total** | 2 709 500 kB (2,6 GiB) |
| **Swap libre** | 2 587 976 kB |
| **Swap utilisé** | 118 MiB |
| **/proc/kcore taille** | 4,0 P (pseudo-taille ELF — ne reflète pas la RAM réelle) |
| **/proc/kcore type** | `ELF 64-bit LSB core file, ARM aarch64` |

## 9.4 Analyse de /proc/kcore

Le pseudo-fichier /proc/kcore est un artefact propre au noyau Linux. Il expose une vue ELF de la mémoire physique du noyau, accessible en lecture seule par l'utilisateur root. Sa taille affichée de 4,0 Pétaoctets est une valeur nominale liée à l'espace d'adressage virtuel 64 bits — elle ne correspond pas à la RAM physique réellement installée.

/proc/kcore peut être utilisé pour des opérations de lecture en mémoire noyau (par exemple avec gdb ou des outils d'analyse), mais il présente des limitations importantes dans un contexte forensic strict :

- Il ne capture pas un état instantané et immuable de la mémoire : la mémoire continue d'évoluer pendant la lecture.
- Il n'est pas portable : le fichier .kcore ne peut pas être copié et analysé sur une autre machine.
- Il ne respecte pas le critère d'atomicité d'une acquisition forensic fiable.
- Sur certaines configurations kernel, des zones mémoire peuvent être inaccessibles ou retourner des zéros.

> **NOTE FORENSIC**
> Pour une acquisition RAM Linux rigoureuse et forensiquement acceptable, l'outil recommandé est LiME (Linux Memory Extractor). LiME fonctionne comme un module noyau chargeable (LKM), ce qui lui permet de capturer la mémoire physique de façon plus atomique que /proc/kcore. Il produit des images aux formats RAW ou LiME, directement analysables avec Volatility.

```
┌──(pat㉿kali)-[~]
└─$ hostnamectl
 Static hostname: kali
       Icon name: computer-vm
         Chassis: vm
      Machine ID: 5b9996b1b0ef4c15a114d72061c751a7
         Boot ID: 27ad4d25608c410babda7ee4f9734090
    AF_VSOCK CID: 1
  Virtualization: qemu
Operating System: Kali GNU/Linux Rolling
          Kernel: Linux 6.16.8+kali-arm64
    Architecture: arm64
 Hardware Vendor: QEMU
  Hardware Model: QEMU Virtual Machine
Hardware Version: virt-10.0
Firmware Version: 0.0.0
   Firmware Date: Fri 2015-02-06
    Firmware Age: 11y 4month 2w 2d
```

```
┌──(pat㉿kali)-[~]
└─$ cat /proc/meminfo | head -30
MemTotal:        4004288 kB
MemFree:          260492 kB
MemAvailable:    3068792 kB
Buffers:          377976 kB
Cached:          2549040 kB
SwapCached:        38752 kB
Active:           583520 kB
Inactive:        2817848 kB
Active(anon):     317684 kB
Inactive(anon):   262180 kB
Active(file):     265836 kB
Inactive(file):  2555668 kB
Unevictable:        9172 kB
Mlocked:             112 kB
SwapTotal:       2709500 kB
SwapFree:        2588004 kB
Zswap:                 0 kB
Zswapped:              0 kB
Dirty:                 0 kB
Writeback:             0 kB
AnonPages:        473364 kB
Mapped:           252276 kB
Shmem:            105732 kB
KReclaimable:     187240 kB
Slab:             251916 kB
SReclaimable:     187240 kB
SUnreclaim:        64676 kB
KernelStack:        7008 kB
PageTables:        14156 kB
SecPageTables:         0 kB
```

```
┌──(pat㉿kali)-[~]
└─$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       911Mi       256Mi       103Mi       3.0Gi       2.9Gi
Swap:          2.6Gi       118Mi       2.5Gi
```

```
┌──(pat㉿kali)-[~]
└─$ ls -lh /proc/kcore
    sudo file /proc/kcore
-r-------- 1 root root 4.0P Jun 23 06:37 /proc/kcore
[sudo] password for pat:
/proc/kcore: ELF 64-bit LSB core file, ARM aarch64, version 1 (SYSV), SVR4-style, from 'BOOT_IMAGE=/boot/vmlinuz-6.16.8+kali-arm64 r
oot=UUID=266f28aa-9139-4acd-b99b-eb'
```

# 10. Acquisition RAM avec DumpIT — Méthode Windows

## 10.1 Présentation de DumpIT

DumpIT est un outil d'acquisition mémoire Windows développé initialement par MoonSols (désormais maintenu par Comae Technologies). Il se distingue par sa simplicité d'utilisation : un seul exécutable portable, aucune installation requise, compatible x86 et x64. Il produit un dump mémoire au format RAW ou DMP, directement analysable avec Volatility.

Sa légèreté en fait un outil idéal pour les opérations de réponse à incident : il peut être placé sur une clé USB et exécuté en quelques secondes sur le système compromis, sans modifier l'état du disque.

## 10.2 Prérequis et procédure

L'exécution de DumpIT nécessite les conditions préalables suivantes :

- Machine cible sous Windows (x86 ou x64 selon la version de DumpIT choisie).
- Droits administrateur sur la machine cible (indispensable pour l'accès à la mémoire physique).
- Espace disque suffisant sur la destination (au moins égal à la RAM totale de la machine).
- Antivirus temporairement désactivé si nécessaire (DumpIT peut être détecté comme suspect).

7. Copier ou déployer l'exécutable DumpIT sur la machine cible (clé USB ou partage réseau).
8. Ouvrir une invite de commande en mode Administrateur.
9. Se déplacer dans le répertoire contenant DumpIT.exe.

10. Lancer l'exécutable — DumpIT demande une confirmation avant de commencer.
11. Valider l'acquisition en appuyant sur Y.
12. Attendre la création du fichier .dmp dans le même répertoire.
13. Calculer le hash SHA256 du fichier .dmp immédiatement après l'acquisition.

**Acquisition mémoire Windows avec DumpIT**

```
REM  Invite de commande Windows — mode Administrateur
cd C:\Users\Analyste\Desktop\DumpIT

REM  Listage du contenu du dossier avant acquisition
dir

REM  Lancement de DumpIT
DumpIt.exe

REM  Calcul du hash SHA256 après acquisition
certutil -hashfile NOMORDINATEUR-YYYYMMDD-HHMMSS.dmp SHA256
```

> ⚠ **POINT D'ATTENTION**
> DumpIT modifie inévitablement une partie de la mémoire en s'exécutant (chargement de son propre code en RAM). C'est une limite inhérente à tout outil d'acquisition côté invité. Cette modification est minimale et documentée, mais elle doit être mentionnée dans le rapport. La méthode hyperviseur (.vmem) ne présente pas cette limitation.

# 11. Acquisition RAM avec FTK Imager — Méthode Windows

## 11.1 Présentation de FTK Imager

FTK Imager est un outil gratuit développé par Exterro (anciennement AccessData). Il offre une interface graphique (GUI) permettant de réaliser des acquisitions mémoire et disque, de monter des images forensics, de calculer des hashes et de visualiser le contenu des images. C'est l'un des outils les plus utilisés dans les formations et les laboratoires forensics Windows.

Pour l'acquisition mémoire, FTK Imager utilise la fonction File > Capture Memory. Elle produit un fichier .mem (image mémoire brute) accompagné optionnellement d'un fichier .ad1 si le pagefile.sys est inclus.

## 11.2 Notion de pagefile.sys

**pagefile.sys :** le fichier de pagination Windows est l'équivalent du swap Linux. Le noyau Windows y déplace les pages mémoire des processus inactifs afin de libérer de la RAM physique. Le pagefile peut contenir des fragments de données sensibles qui ne sont plus présentes en RAM physique : parties de fichiers ouverts, fragments de mots de passe, données de sessions, etc.

L'inclusion du pagefile.sys dans l'acquisition mémoire est donc forensiquement intéressante : elle permet d'accéder à des données qui auraient été swappées hors de la RAM physique. FTK Imager propose cette option lors de la capture mémoire.

### 11.3 Notion de conteneur AD1

**Format AD1 :** le format AD1 (AccessData Custom Content Image) est un conteneur forensic propriétaire FTK. Il permet de stocker une collection de fichiers avec leurs métadonnées, leurs hashes et leurs attributs dans un seul fichier conteneur. Contrairement à une image brute (RAW), AD1 ne capture pas un volume entier mais une sélection de fichiers ou dossiers. Il est utilisé pour les acquisitions logiques (logical acquisition) ou la collecte ciblée de fichiers.

### 11.4 Procédure d'acquisition RAM avec FTK Imager

14. Ouvrir FTK Imager en mode Administrateur.
15. Naviguer dans le menu File > Capture Memory.
16. Sélectionner le dossier de destination pour le fichier .mem.
17. Choisir si le pagefile.sys doit être inclus dans la capture (recommandé).
18. Lancer l'acquisition — une barre de progression affiche l'avancement.
19. Une fois terminée, noter le chemin et le nom du fichier .mem produit.
20. Calculer le hash SHA256 du fichier .mem (FTK Imager peut le faire directement).

## 12. Acquisition disque — Équivalent dd sur Kali

### 12.1 Contexte et justification de l'équivalent Linux

> **ADAPTATION LINUX / KALI**
> L'acquisition disque avec FTK Imager est documentée procéduralement dans ce rapport (section 12.2). La réalisation pratique a été effectuée avec dd sur Kali Linux, qui constitue l'équivalent open source direct de FTK Imager pour l'acquisition d'images RAW. Le résultat est fonctionnellement identique : une copie bit-à-bit de la partition source.

L'analyse préalable du support avec lsblk a permis d'identifier la topologie des disques disponibles sur la machine de test :

**Identification des disques**

```
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINT,MODEL
```

```
NAME   SIZE  FSTYPE  MOUNTPOINT
vda    50G
├─vda3  48G  ext4    /
└─vda4   2G  swap    [SWAP]
vdb     1G
└─vdb1 1023M  vfat   /home/pat/carving_test
```

La partition /dev/vdb1 est un support de test dédié, formaté en FAT32, clairement séparé du disque système /dev/vda. Son montage en /home/pat/carving_test confirme son usage comme support d'expérimentation pour le TP.

## 12.2 Procédure FTK Imager — Documentation théorique

La procédure d'acquisition disque avec FTK Imager suit les étapes ci-dessous. FTK Imager supporte plusieurs types de sources et plusieurs formats de sortie.

21. Ouvrir FTK Imager en mode Administrateur.
22. Sélectionner File > Create Disk Image.
23. Choisir le type de source : Physical Drive (disque physique), Logical Drive (partition), Image File (image existante), ou Contents of a Folder.
24. Sélectionner la source dans la liste des disques et partitions détectés.
25. Cliquer sur Add pour ajouter une destination et choisir le format de sortie : RAW (.img/.dd), E01, AFF, ou AD1.
26. Renseigner les métadonnées de l'évidence : nom de l'analyste, numéro de cas, description.
27. Lancer l'acquisition. FTK Imager affiche la progression et calcule les hashes MD5 et SHA1 en temps réel.
28. À la fin, vérifier les hashes affichés dans le résumé final (Computed Hash = Stored Verification Hash = Verify Result).

| | |
|---|---|
| **Type de source choisi** | Partition logique (/dev/vdb1) |
| **Format de sortie** | RAW (équivalent dd) |
| **Nom de l'image** | `kali_partition.img` |
| **Chemin de sortie** | `~/Desktop/EVIDENCES_PART_I.I/kali_partition.img` |

## 12.3 Acquisition pratique avec dd

L'acquisition est réalisée avec dd, configuré avec les paramètres forensics habituels détaillés ci-dessous :

**Acquisition forensic avec dd**

```
# Création du répertoire d'évidences
mkdir -p ~/Desktop/EVIDENCES_PART_I.I

# Acquisition forensic bit-à-bit de /dev/vdb1
sudo dd if=/dev/vdb1 \
        of=~/Desktop/EVIDENCES_PART_I.I/kali_partition.img \
        bs=4M status=progress conv=noerror,sync
```

```
255+1 records in
256+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied
```

Interprétation du résultat : 255+1 records in indique que dd a lu 255 blocs complets de 4 Mo et 1 bloc partiel — ce comportement est normal en fin de partition. Les 256+0 records out signifient que 256 blocs complets ont été écrits dans l'image, le bloc partiel ayant été complété par des zéros grâce au paramètre conv=sync. L'image finale fait exactement 1 073 741 824 octets.

## 12.4 Vérification du type et du hash de l'image

**Vérification de l'image forensic**

```
# Vérification du type de l'image
file ~/Desktop/EVIDENCES_PART_I.I/kali_partition.img

# Calcul et archivage du hash SHA256
sha256sum ~/Desktop/EVIDENCES_PART_I.I/kali_partition.img | tee
~/Desktop/EVIDENCES_PART_I.I/kali_partition.sha256

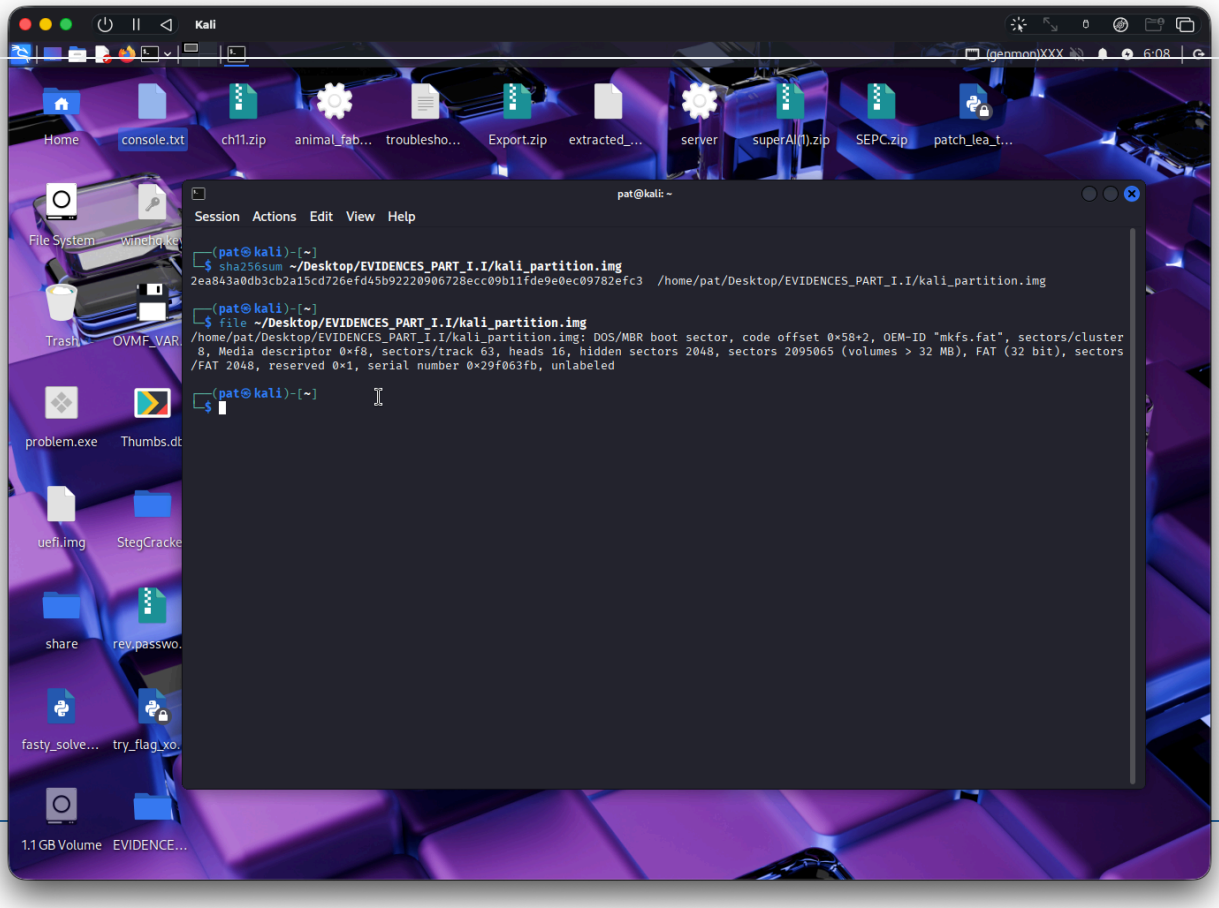
# Vérification de la taille
ls -lh ~/Desktop/EVIDENCES_PART_I.I/kali_partition.img
```

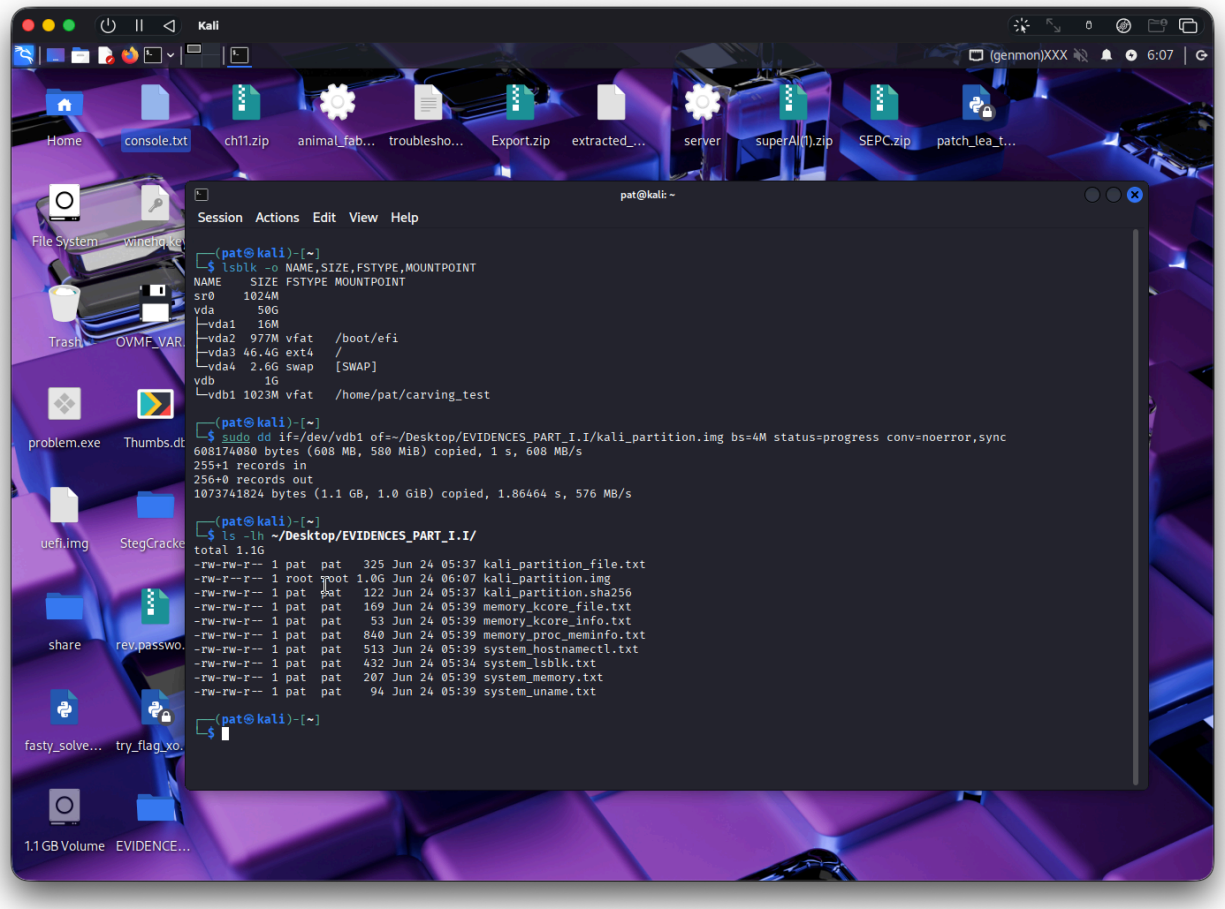
```
kali_partition.img: DOS/MBR boot sector; OEM-ID "mkfs.fat", FAT (32 bit), ...

2ea843a0db3cb2a15cd726efd45b92220906728ecc09b11fde9e0ec09782efc3
kali_partition.img
```

La commande file confirme que l'image est bien une image FAT32 valide, contenant un secteur de boot MBR correct. Le type détecté (DOS/MBR boot sector, OEM-ID "mkfs.fat", FAT 32 bit) est cohérent avec la partition source /dev/vdb1 formatée en vfat/FAT32.

| | |
|---|---|
| **Image acquise** | ~/Desktop/EVIDENCES_PART_I.I/kali_partition.img |
| **Taille** | 1 073 741 824 octets (1,0 GiB) |
| **Type détecté** | DOS/MBR boot sector — mkfs.fat — FAT 32 bit |
| **Hash SHA256** | 2ea843a0db3cb2a15cd726efd45b92220906728ecc09b11fde9e0ec09782efc3 |






# 13. Vérification d'intégrité des images forensics

## 13.1 Les trois hashes de FTK Imager

À la fin d'une acquisition disque avec FTK Imager, l'outil affiche un résumé de vérification contenant trois valeurs de hash distinctes. La compréhension de ces trois valeurs est essentielle pour valider l'intégrité de l'image produite.

| | |
|---|---|
| **Computed Hash** | Hash calculé par FTK Imager sur l'image produite, immédiatement après l'écriture. C'est la valeur de référence principale de l'acquisition. |
| **Stored Verification Hash** | Hash stocké directement dans le fichier image (dans les métadonnées pour E01, ou recalculé sur le fichier RAW). Il est comparé au Computed Hash pour valider l'écriture. |
| **Verify Result** | Hash issu du rapport de cas généré à côté de l'image. Il représente la vérification finale indépendante. |

Les trois hashes d'une même famille algorithmique (MD5, SHA1 ou SHA256) doivent être strictement identiques. Si ces valeurs sont identiques, l'image a été créée correctement : elle correspond bit-à-bit à la source acquise et son intégrité est préservée. Toute divergence indique une corruption ou une altération de l'image.

## 13.2 Validation de l'image kali_partition.img

| | |
|---|---|
| **SHA256 — Calculé lors de l'acquisition** | `2ea843a0db3cb2a15cd726efd45b92220906728ecc09b11fde9e0ec09782efc3` |
| **SHA256 — Vérification post-acquisition** | `2ea843a0db3cb2a15cd726efd45b92220906728ecc09b11fde9e0ec09782efc3` |

✓ **HASHES IDENTIQUES — Intégrité cryptographique confirmée**

✓ **RÉSULTAT VALIDÉ**

Les deux hashes SHA256 sont identiques. L'image kali_partition.img est une copie bit-à-bit fidèle et vérifiée de la partition /dev/vdb1. L'intégrité de l'image est cryptographiquement garantie.

# 14. Parties optionnelles

## 14.1 RedLine RAM — OPTIONNEL

RedLine est un outil gratuit développé par Mandiant (anciennement FireEye) permettant de réaliser une collecte d'informations forensic complète sur un système Windows, incluant optionnellement une acquisition mémoire. Il est particulièrement orienté vers la détection de malwares et l'analyse d'incidents de sécurité.

RedLine se distingue des outils d'acquisition pure par ses capacités d'analyse intégrée : il peut détecter les hooks IDT/SSDT (techniques de rootkits), calculer des hashes MD5 sur les sections mémoire (MemD5), analyser les processus, les connexions réseau et les artefacts de registre.

| | |
|---|---|
| **Statut** | OPTIONNEL — Non réalisé dans cette adaptation |
| **Hook Detection** | Recherche de hooks IDT/SSDT — techniques de rootkit/malware |
| **MemD5** | Hash MD5 appliqué à des sections de mémoire pour contrôle d'intégrité |
| **Format de sortie** | Sessions d'analyse / AnalysisSession |

## 14.2 Shadow Copy / VSS Windows — OPTIONNEL

Le Volume Shadow Copy Service (VSS) est un service Windows qui crée des clichés instantanés (Shadow Copies) du volume disque à intervalles réguliers ou lors d'événements système. Ces snapshots permettent de retrouver des versions antérieures de fichiers, même si ceux-ci ont été supprimés ou modifiés depuis.

Dans un contexte forensic, les Shadow Copies sont particulièrement intéressantes pour récupérer des fichiers supprimés ou chiffrés par un ransomware, à condition que les VSS n'aient pas été effacés (ce que font souvent les ransomwares modernes en priorité).

**Manipulation des Shadow Copies Windows**

```
REM  Lister les Shadow Copies disponibles
vssadmin list shadows
```

```
REM  Créer un lien symbolique vers une Shadow Copy spécifique
mklink /D C:\monVSS \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopyX\

REM  Naviguer dans la Shadow Copy comme un répertoire normal
dir C:\monVSS\
```

| | |
|---|---|
| **Statut** | OPTIONNEL — Non réalisé |
| **Outil cité** | ShadowCopyView de NirSoft (visualisation graphique des VSS) |

## 14.3 Image disque VMDK — OPTIONNEL

Les fichiers .vmdk représentent les disques virtuels des machines VMware. Ils peuvent être analysés directement comme images forensics sans nécessiter d'acquisition depuis la machine invitée. Cette approche présente l'avantage de ne pas interagir avec le système d'exploitation de la VM.

**✓ Avantages**

- Pas d'acquisition nécessaire depuis la VM
- Accès direct au disque virtuel depuis l'hôte
- Montage possible dans Autopsy ou via qemu-nbd
- Fonctionne même si la VM ne peut plus démarrer

**✗ Inconvénients**

- Risque de modification si monté en écriture
- Gestion des snapshots complexe (fichiers delta)
- Volumes importants (plusieurs dizaines de Go)
- Nécessite des outils adaptés (qemu-img, virt-manager)

| | |
|---|---|
| **Statut** | OPTIONNEL — Non réalisé |

# 15. Synthèse générale et chaîne de conservation

## 15.1 Bilan des acquisitions réalisées

| | Acquisition | Outil | Résultat | Artefact / Remarque |
|---|---|---|---|---|
| 1 | Image VMEM (VMware) | VMware Suspend | Non réalisé | Outil VMware non disponible — adapté sur Kali |
| 2 | RAM avec DumpIT | DumpIT.exe | Non réalisé | Outil Windows — documenté théoriquement |
| 3 | RAM avec FTK Imager | FTK Imager | Non réalisé | Outil Windows — documenté théoriquement |
| 4 | Observation mémoire | /proc/kcore + free | **Réalisé ✓** | État système documenté — Kali arm64 |
| 5 | Image disque (partition) | dd (équiv. FTK) | **Réalisé ✓** | kali_partition.img — 1,0 GiB — SHA256 vérifié |

## 15.2 Chaîne de conservation des preuves

Les étapes suivantes constituent la chaîne de conservation documentée pour les artefacts produits dans ce TP :

| | |
|---|---|
| **Identification** | Support /dev/vdb1 identifié par lsblk avant toute manipulation |
| **Isolation** | Support de test (/dev/vdb1) distinct du disque système (/dev/vda) |
| **Acquisition** | Image bit-à-bit avec dd — conv=noerror,sync — aucun accès au support original pendant l'analyse |
| **Hachage** | SHA256 calculé immédiatement après l'acquisition et archivé dans le dossier d'évidences |
| **Conservation** | Image stockée dans ~/Desktop/EVIDENCES_PART_I.I/ — accès contrôlé |
| **Documentation** | Chaque commande notée avec ses paramètres, résultats et timestamps |
| **Vérification** | Hash recalculé en fin d'analyse pour confirmer l'absence de modification |

## 15.3 Récapitulatif des artefacts produits

| Artefact | Taille | Hash SHA256 |
|---|---|---|
| `kali_partition.img` | 1 073 741 824 octets (1,0 GiB) | `2ea843a0...782efc3` |
| `kali_partition.sha256` | < 1 Ko | Fichier de référence — contient le hash SHA256 |
| `system_hostnamectl.txt / uname.txt / memory.txt` | < 1 Ko chacun | Documentation système — phase d'observation |

# 16. Conclusion et enseignements

## 16.1 Bilan de l'investigation

Ce TP a permis d'explorer les deux grandes familles d'acquisition forensic : la mémoire vive (RAM) et la mémoire de masse (disque). Ces deux types d'acquisitions sont complémentaires et constituent ensemble la base de toute investigation forensic sérieuse.

L'adaptation de l'environnement (Kali Linux / QEMU / arm64 en lieu et place de Windows) a nécessité des ajustements méthodologiques, mais elle a également permis d'approfondir la compréhension des mécanismes sous-jacents : fonctionnement de /proc/kcore, comportement de dd, structure des systèmes de fichiers FAT32, et calcul de hashes cryptographiques en ligne de commande.

## 16.2 Enseignements principaux

- **Volatilité de la mémoire :** la RAM doit être acquise en priorité lors d'une intervention sur un système actif — chaque minute d'attente peut signifier la perte d'artefacts critiques (connexions réseau, processus, clés de chiffrement).
- **Supériorité de l'acquisition hyperviseur :** pour les machines virtuelles, l'acquisition via hyperviseur (.vmem, snapshot) est préférable à l'exécution d'un outil dans la VM car elle est plus atomique et ne modifie pas l'état de la mémoire invitée.
- **Universalité de dd :** dd est l'outil d'acquisition disque de référence sous Linux. Sa simplicité, sa flexibilité et sa disponibilité sur tous les systèmes Unix/Linux en font un incontournable de la forensic open source.
- **Obligatoire — les hashes :** calculer et archiver le hash SHA256 de chaque artefact dès sa création est non négociable. Sans hash de référence, il est impossible de prouver l'intégrité d'une preuve numérique.
- **Limites de /proc/kcore :** bien que /proc/kcore expose la mémoire noyau Linux, il ne constitue pas une acquisition forensic valide au sens strict. LiME est la solution appropriée pour une acquisition RAM Linux rigoureuse.
- **Documentation systématique :** chaque commande exécutée, chaque résultat obtenu et chaque décision de l'analyste doivent être consignés. Un rapport sans documentation est un rapport sans valeur probatoire.

## 16.3 Points à améliorer et perspectives

Dans un contexte professionnel réel ou pour une itération future de ce TP, les points suivants mériteraient d'être approfondis :

- Réaliser une acquisition RAM Linux avec LiME et l'analyser avec Volatility pour compléter la partie mémoire.
- Produire une image au format E01 (avec ewfacquire ou FTK Imager) pour comparer les avantages du format forensic structuré par rapport au RAW.
- Effectuer les parties optionnelles RedLine, Shadow Copy et VMDK pour avoir une vision complète des techniques d'acquisition.
- Automatiser la chaîne d'acquisition avec un script shell qui documente l'environnement, réalise l'acquisition et calcule les hashes en une seule opération.

# Annexe A — Lexique forensic

**Acquisition forensic :** Processus de création d'une copie fidèle et vérifiable d'un support numérique (disque, RAM, partition) en vue d'une analyse ultérieure. L'acquisition doit être documentée, hachée et réalisée sur une copie, jamais sur l'original.

**AD1 :** Format de conteneur forensic propriétaire AccessData/Exterro. Stocke une collection de fichiers avec leurs métadonnées et hashes dans un fichier unique. Utilisé par FTK Imager pour les acquisitions logiques.

**Atomicité :** Critère de fiabilité d'une acquisition mémoire : l'image doit refléter un état cohérent de la RAM, non altéré par des modifications concurrentes pendant la lecture.

**Chain of custody :** Chaîne de conservation documentée des preuves numériques. Elle trace le parcours d'une preuve depuis sa collecte jusqu'à sa présentation, garantissant qu'elle n'a pas été modifiée, perdue ou compromise.

**dd :** Outil Linux de copie bit-à-bit (disk duplicator). Standard de facto pour l'acquisition forensic de disques et partitions sur systèmes Unix/Linux. Paramètres clés : if (source), of (destination), bs (taille de bloc), conv=noerror,sync.

**DMP :** Format de dump mémoire Windows. Produit par DumpIT ou lors de crashes système (Blue Screen). Contient des en-têtes structurés analysables par Volatility.

**E01 / EWF :** Expert Witness Format — format d'image disque forensic de référence. Intègre compression, métadonnées de cas, hashes de vérification et découpage en segments. Produit par FTK Imager et EnCase.

**Fileless malware :** Malware qui s'exécute entièrement en mémoire RAM sans écrire de fichier sur le disque. Invisible aux antivirus basés sur les fichiers. Détectable uniquement par analyse mémoire (Volatility).

**Hash / SHA256 :** Empreinte cryptographique d'un fichier. SHA256 produit une valeur de 256 bits (64 caractères hexadécimaux). Deux fichiers identiques ont nécessairement le même hash. Toute modification, même d'un bit, produit un hash radicalement différent.

**Hook IDT/SSDT :** Technique de rootkit consistant à modifier les tables de dispatch du noyau (IDT = Interrupt Descriptor Table, SSDT = System Service Descriptor Table) pour intercepter les appels système. Détectable par RedLine et certains modules Volatility.

**LiME :** Linux Memory Extractor — module noyau (LKM) permettant une acquisition RAM Linux plus atomique que /proc/kcore. Produit des images RAW ou LiME directement analysables avec Volatility.

**Pagefile.sys :** Fichier de pagination Windows (équivalent du swap Linux). Contient les pages mémoire des processus inactifs. Peut contenir des fragments de données sensibles et doit être inclus dans l'acquisition forensic.

**RAW :** Format d'image brute : copie bit-à-bit sans en-tête ni métadonnées. Simple, universel, compatible avec tous les outils forensics. Produit par dd (Linux) ou FTK Imager (Windows).

**VMEM :** Fichier généré par VMware lors de la mise en pause d'une VM ou d'un snapshot avec mémoire. Représente l'état exact de la RAM de la machine virtuelle. Analysable directement avec Volatility.

**Volatility :** Framework open source d'analyse de mémoire post-acquisition. Ne réalise pas d'acquisition : il analyse des images mémoire déjà créées (RAW, DMP, VMEM, LiME). Permet d'extraire processus, connexions réseau, artefacts de registre, etc.

**VSS / Shadow Copy :** Volume Shadow Copy Service — mécanisme Windows de création de clichés instantanés du disque. Permet de retrouver des fichiers supprimés ou modifiés via des instantanés antérieurs.

# Annexe B — Commandes de référence

## Linux / Kali

**Commandes Linux essentielles**

```
# Identification du système
hostnamectl ; uname -a

# État de la mémoire
free -h
cat /proc/meminfo | head -30
ls -lh /proc/kcore ; sudo file /proc/kcore

# Identification des disques
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINT,MODEL

# Acquisition disque bit-à-bit
sudo dd if=/dev/PARTITION of=/chemin/image.img bs=4M status=progress
conv=noerror,sync

# Calcul de hash
sha256sum fichier.img | tee fichier.img.sha256
md5sum fichier.img
sha1sum fichier.img

# Vérification du type d'une image
file image.img

# Vérification de la taille
ls -lh image.img
du -sh image.img
```

## Windows

**Commandes Windows essentielles**

```
REM  Calcul de hash
certutil -hashfile FICHIER SHA256
certutil -hashfile FICHIER MD5
certutil -hashfile FICHIER SHA1

REM  PowerShell — hash
Get-FileHash -Path FICHIER -Algorithm SHA256

REM  Listage de fichiers
dir /B /S C:\chemin\

REM  DumpIT — acquisition RAM
DumpIt.exe

REM  Shadow Copies
vssadmin list shadows
mklink /D C:\monVSS \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopyX\
```

## macOS / hôte VMware

**Commandes macOS**

```
# Hash SHA256
shasum -a 256 fichier.vmem
md5 fichier.vmem

# Listage des fichiers VM
ls -lh /chemin/vers/VM/
```

*— Fin du rapport —*